



**UNIVERSITATEA
TEHNICĂ**
DIN CLUJ-NAPOCA

Bug Reporter

Assignement 2

Autori: Cimpan Alex-Ciprian, Ilco Doina-Cezara, Marcu Andrei
Grupa: 30233

FACULTATEA DE AUTOMATICA
SI CALCULATOARE

Martie 2026

Cuprins

1	Introducere	2
2	Tech stack	2
3	Arhitectură	2
4	Modelul Entităților	4
4.1	Diagrama Entităților	4
4.2	Enum-ul BugStatus	4
4.3	Relațiile dintre entități	5
5	Frontend	5
5.1	Structura Paginilor	5
5.2	Structura Componentelor	6
6	Securitate	6
7	Testare	7
7.1	Teste de Repository	7
7.2	Teste de Service	7
8	Baza de Date	8
8.1	Configurare	8
8.2	Tabele generate automat	8

1 Introducere

BugReporter este o aplicatie web care permite utilizatorilor sa raporteze, sa urmareasca si sa gestioneze bug-uri software. Utilizatorii pot crea rapoarte de bug-uri cu tag-uri, pot comenta bug-urile existente si pot efectua operatii CRUD complete pe toate entitatile. Aplicatia include un sistem de votare (upvote/downvote) pe bug-uri si comentarii, cu calcul automat al scorului de reputatie al utilizatorilor.

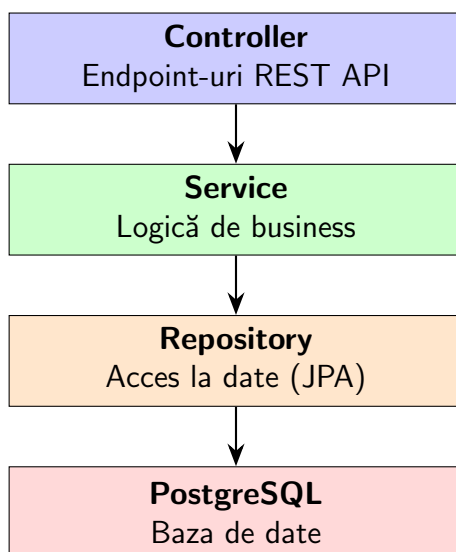
2 Tech stack

Tabela 1: Tehnologiile utilizate in proiect

Tehnologie	Versiune	Scop
Java	17	Limbaaj de programare
Spring Boot	4.0.3	Framework principal
Spring Data JPA	–	Acces la date / ORM
Hibernate	–	Implementare JPA
Spring Security	–	Criptarea parolelor (BCrypt)
PostgreSQL	–	Baza de date relationala
Lombok	–	Reducerea codului boilerplate
JUnit 5	–	Framework de testare unitara
Mockito	–	Framework de mock-uri pentru teste
Maven	–	Build si gestionarea dependentelor
React	19.2.4	Framework UI frontend
React Router DOM	7.13.2	Rutare client-side (SPA)
Vite	5.4.0	Build tool si dev server frontend

3 Arhitectură

Aplicația urmează o **Arhitectură pe Straturi** (3 niveluri), care separă responsabilitățile în straturi distincte:



- **Controller** – Gestionează cererile și răspunsurile HTTP. Adnotat cu `@RestController`.
- **Service** – Conține logica de business. Adnotat cu `@Service`.
- **Repository** – Interfețe care extind `JpaRepository` pentru operațiile cu baza de date.
- **Entity** – Entități JPA mapate pe tabelele din PostgreSQL folosind `@Entity`.
- **Config** – Configurarea securității (`SecurityConfig` pentru regulile HTTP, `PasswordEncoderConfig` pentru BCrypt).

Comunicarea dintre frontend (React, port 5173) și backend (Spring Boot, port 8080) se realizează prin **REST API** cu date în format **JSON**.

4 Modelul Entităților

4.1 Diagrama Entităților

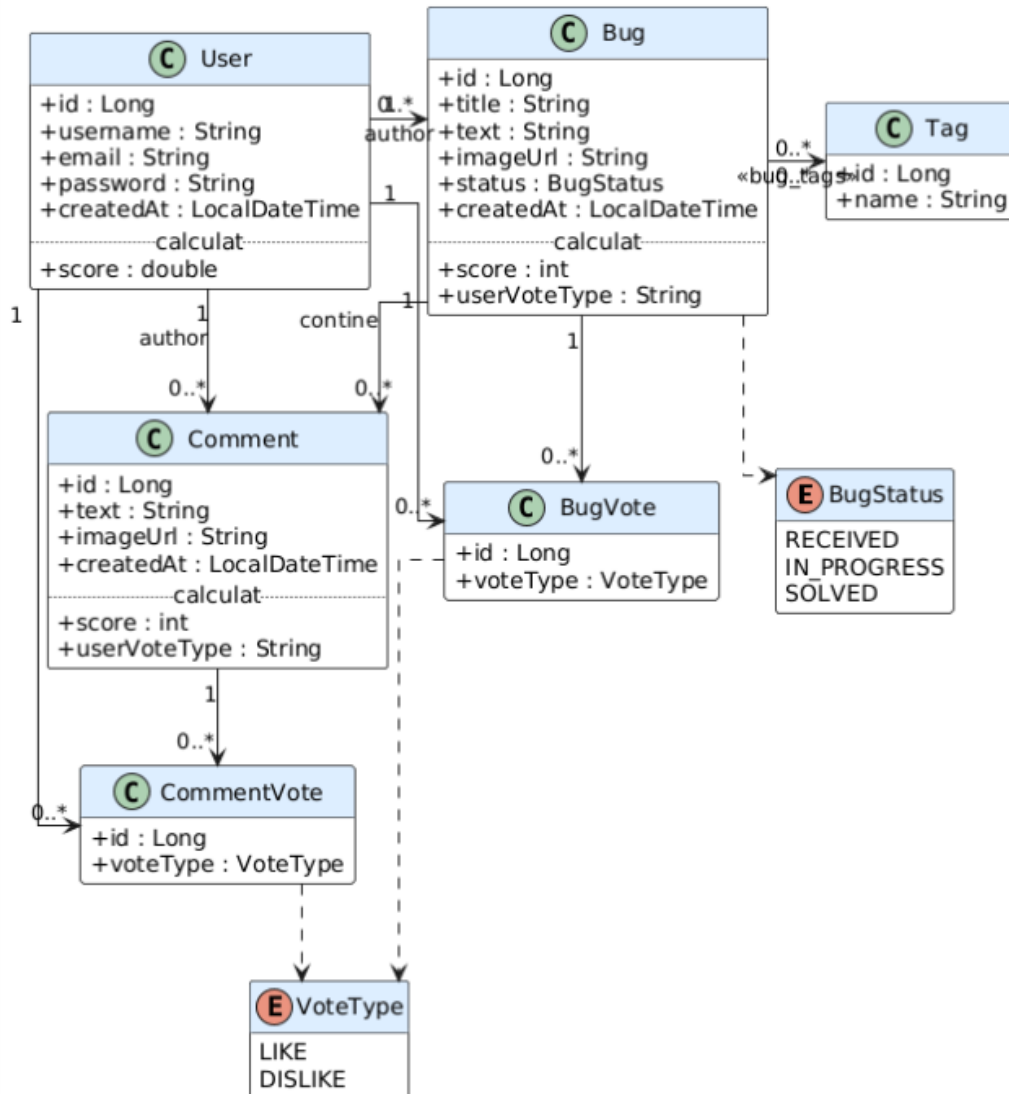


Figura 1: Entity Diagram

4.2 Enum-ul BugStatus

Bug-urile pot avea unul din următoarele statusuri:

```
1 public enum BugStatus {  
2     RECEIVED, // Primit - cand utilizatorul posteaza bug-ul
```

```

3     IN_PROGRESS, // In progres - la primul comentariu
4     SOLVED      // Rezolvat - marcat de autor
5 }

```

Listing 1: BugStatus — statusul unui bug

```

1 public enum VoteType {
2     LIKE, // Upvote / vot pozitiv
3     DISLIKE // Downvote / vot negativ
4 }

```

Listing 2: VoteType — tipul votului

4.3 Relațiile dintre entități

- **User** → **Bug**: Un utilizator poate fi autorul mai multor bug-uri (**one-to-many**).
- **User** → **Comment**: Un utilizator poate fi autorul mai multor comentarii (**one-to-many**).
- **Bug** → **Comment**: Un bug poate avea mai multe comentarii (**one-to-many**).
- **Bug** ↔ **Tag**: Un bug poate avea mai multe tag-uri și un tag poate fi asociat mai multor bug-uri (**many-to-many**), prin tabela de legătură `bug_tags`.
- **BugVote**: Leaga un utilizator de un bug cu un tip de vot. Constrangere unica pe perechea (`user_id`, `bug_id`).
- **CommentVote**: Leaga un utilizator de un comentariu cu un tip de vot. Constrangere unica pe perechea (`user_id`, `comment_id`).

5 Frontend

5.1 Structura Paginilor

Aplicatia frontend este o SPA (Single Page Application) construita cu React si React Router DOM. Rutele sunt protejate — utilizatorii neautentificati sunt redirectionati catre pagina de Login.

- **Login / Register** — pagini de autentificare si inregistrare
- **Bugs** — lista tuturor bug-urilor cu filtre (tag, titlu, autor, my bugs)
- **BugDetail** — detalii bug, comentarii sortate dupa scor, voturi, marcare ca rezolvat

- **CreateBug / EditBug** — formulare creare si editare bug cu upload imagine
- **Profile** — profilul utilizatorului cu statistici si lista bug-urilor proprii

5.2 Structura Componentelor

```

1 src/
2     components/
3         Navbar/           # Bara de navigare reutilizabila
4     pages/
5         Login/            # Autentificare
6         Register/         # Inregistrare
7         Bugs/             # Lista bug-uri cu filtre
8         BugDetail/        # Detalii + comentarii + voturi
9         CreateBug/        # Formular creare
10        EditBug/          # Formular editare
11        Profile/          # Profil utilizator
12    api.js                # Toate apelurile HTTP (centralizat)
13    App.jsx               # Routing si stare globala
14    main.jsx              # Punct de intrare

```

Listing 3: Structura directoarelor Frontend

Starea utilizatorului autentificat este pastrata in `localStorage` si accesata global prin `App.jsx`.

6 Securitate

Securitatea aplicației este configurată în două clase separate:

- **PasswordEncoderConfig** – Definește bean-ul `BCryptPasswordEncoder` pentru criptarea parolelor.
- **SecurityConfig** – Configurează regulile HTTP: dezactivează CSRF și permite accesul la toate endpoint-urile.

Fluxul de criptare a parolei:

1. Utilizatorul trimite parola în text clar prin request POST/PUT.
2. `UserService` apelează `passwordEncoder.encode()` înainte de salvare.
3. Parola este stocată în baza de date ca hash BCrypt (ex: `$2a$10$N9qo8uL0...`).
4. BCrypt este un algoritm unidirecțional – parola nu poate fi decriptată, doar verificată cu `passwordEncoder.matches()`.

7 Testare

Proiectul include două niveluri de testare:

7.1 Teste de Repository

Teste de integrare folosind `@DataJpaTest` cu o bază de date PostgreSQL dedicată pentru teste (`bugtracker.test`). Profilul de test este configurat în `application-test.properties` cu `ddl-auto=create` pentru o bază de date curată la fiecare rulare.

- `UserRepositoryTest` – salvare utilizator, căutare după username, căutare după email
- `BugRepositoryTest` – salvare bug cu autor, salvare bug fără autor (eșec așteptat)
- `CommentRepositoryTest` – salvare comentariu cu autor și bug, căutare după bugId
- `TagRepositoryTest` – constrângere de unicitate pe nume, căutare după nume

7.2 Teste de Service

Teste unitare folosind **Mockito** pentru a simula dependențele de repository.

- `UserServiceTest` – creare (cu criptarea parolei), obținere toți, obținere după ID, actualizare (cu re-criptarea parolei), ștergere, excepție la negăsit
- `BugServiceTest` – creare (status setat automat la RECEIVED), obținere toți, obținere după ID, actualizare, ștergere, excepție la negăsit
- `CommentServiceTest` – creare, obținere toți, obținere după ID, obținere după bugId, actualizare, ștergere, excepție la negăsit
- `BugVoteServiceTest` — logica vot (toggle, schimbare tip, self-vote blocat)
- `UserScoreServiceTest` — formula de calcul a scorului de reputatie

8 Baza de Date

8.1 Configurare

Aplicația folosește două baze de date PostgreSQL:

- **bugtracker** – baza principală, folosită în dezvoltare și producție
- **bugtracker_test** – baza de test, cu `ddl-auto=create` (tabelele se recreează la fiecare rulare a testelor)

8.2 Tabele generate automat

Hibernate generează automat următoarele tabele pe baza entităților JPA:

- **users** – utilizatori (id, username, email, password, created_at)
- **bug** – bug-uri (id, title, text, image_url, status, created_at, author_id)
- **comment** – comentarii (id, text, image_url, created_at, author_id, bug_id)
- **tag** – tag-uri (id, name)
- **bug_tags** – tabela de legătură ManyToMany între bug și tag (bug_id, tag_id)